

SPEAKER SCRIPT · 발표 대본

WEATHER FIT TALK

날씨 × 패션 AI 스타일링 챗봇 — 슬라이드별 발표 스크립트

SPEAKER 박수용 SLIDES 총 15장 DURATION 약 6-8분

이 문서는 **슬라이드 한 장당 20~40초** 분량의 발표 대본입니다. 어려운 용어는 괄호로 풀어 설명을 붙였습니다. 자연스럽게 읽되, **굵게 표시된 부분**은 특히 강조해서 말씀해 주세요.

SLIDE 01

표지 · WEATHER FIT TALK

▶ 표지가 떠 있는 상태에서 — 자연스럽게 인사부터

안녕하세요, 오늘 발표를 맡은 **박수용**입니다.

제가 이번에 만든 프로젝트 이름은 **WEATHER FIT TALK**, 한 줄로 말씀드리면 **날씨와 패션을 결합한 AI 스타일링 챗봇**입니다.

특히 **로컬 LLM** (= 인터넷 서버가 아니라 내 컴퓨터 안에서 직접 돌아가는 AI) 을 활용했다는 점이 가장 큰 특징인데요, 지금부터 어떤 프로젝트인지 차근차근 소개해드리겠습니다.

TIP 첫 슬라이드는 천천히. 청중과 눈을 한 번 맞추고 시작하세요.

SLIDE 02

발표 개요 — 오늘의 순서

▶ 네 개의 챕터 카드를 왼쪽부터 차례대로 가리키며

본격적인 소개에 앞서, 오늘 발표가 어떻게 진행될지 전체 흐름을 먼저 보여드리겠습니다.

오늘은 크게 **네 부분**으로 나눠서 말씀드리겠습니다. 첫 번째는 **INTRO** — 프로젝트가 무엇이고 왜 만들었는지, 그리고 어떤 기능이 있는지 소개합니다. 두 번째는 **STRUCTURE** — 사용한 기술과 전체 동작 흐름, 파일 구조를 살펴봅니다.

세 번째가 오늘 발표의 핵심인 **CODE** 파트입니다. **app.py · prompts.py · tools.py** 세 파일의 역할을 하나씩 풀어드리고요. 마지막 **CLOSING** 에서 실행 방법과 앞으로의 계획으로 마무리하겠습니다.

TIP "세 번째 CODE 파트가 오늘 발표의 핵심" — 청중에게 어디에 집중하면 되는지 미리 알려주세요.

▶ 채팅 예시 화면을 가리키며

이 챗봇은 **오늘 날씨와 상황에 맞는 코디를 AI가 추천해주는 서비스**입니다.

예를 들어 사용자가 "**오늘 비 오고 바람이 불어. 학교 갈 때 뭐 입을까?**" 하고 물어보면, AI가 상의·하의·아우터·신발·소품까지 한 번에 정리해서 추천해주는 구조입니다.

그러니까 단순히 날씨만 알려주는 앱이 아니라, **그 날씨에 맞는 옷차림까지 같이 제안해준다**는 점이 가장 큰 차별점이라고 보시면 됩니다.

▶ 4개의 카드를 차례대로 짚으며

그럼 왜 이런 걸 만들게 됐는지 말씀드리겠습니다.

사실 매일 아침에 옷 고르는 게 은근히 귀찮은 일입니다. 기존 날씨 앱은 기온이나 강수확률만 알려줄 뿐, **구체적으로 뭘 입어야 하는지는 알려주지 않습니다.**

그래서 두 가지를 시도해보고 싶었습니다. 첫째, **AI를 활용해서 날씨와 상황을 함께 고려한 추천**을 만들어보자. 그리고 둘째, 내 컴퓨터에서 직접 돌아가는 **로컬 LLM의 구조를 직접 이해해보자**. 이 두 가지가 큰 동기였습니다.

▶ 모든 카드를 일일이 읽지 말고, 카테고리로 묶어서

핵심 기능은 크게 여덟 가지인데요, 한꺼번에 읽으면 너무 많으니 묶어서 설명드리겠습니다.

우선 **날씨·상황·스타일** 세 가지 축으로 추천을 합니다. 비 오는 날인지, 등교인지 면접인지, 미니멀인지 스트릿인지를 함께 고려합니다.

그리고 **사용자가 직접 쓴 문장을 사이드바 조건보다 먼저 반영**합니다. 필요할 때는 **DuckDuckGo** (= 구글 같은 검색 엔진) 검색 결과까지 활용하고요, **인터넷 없이도 내 PC에서 바로 돌아간다는 점**도 큰 장점입니다.

▶ 핵심 3개만 짚어서 (OLLAMA · GEMMA · STREAMLIT)

사용한 기술은 전부 오픈소스입니다. 전체 로직은 **파이썬**으로 짰고, 웹 화면은 **Streamlit** (= 파이썬 코드로 웹 페이지를 만들 수 있는 도구) 으로 빠르게 만들었습니다.

가장 중요한 건 **Ollama**인데요, 쉽게 말해 **내 컴퓨터에 AI 모델을 깔아두고 직접 돌리는 도구**입니다. 그 위에 구글이 공개한 **Gemma 4:e4b** 모델을 올려서 답변을 생성합니다.

나머지는 DuckDuckGo 검색이나 통신 라이브러리처럼 보조적인 도구들이라고 보시면 됩니다.

▶ 화살표를 따라 위에서 아래로 손가락으로 짚으며

이 슬라이드가 오늘 발표에서 가장 중요한 슬라이드입니다. **사용자가 질문하면 어떻게 답변이 나오는** 지를 한눈에 보여주는 흐름도입니다.

사용자가 질문을 입력하면, 우선 **Streamlit 화면**에서 받은 입력이 **tools.py**로 넘어가 **날씨와 스타일을 정리**합니다. 그 다음 **prompts.py**에서 만든 답변 규칙과 합쳐서 **Ollama 로컬 LLM**에 전달됩니다.

그러면 LLM이 답변을 생성하고, 그 답변이 다시 Streamlit 화면에 출력됩니다. 이 흐름만 기억하시면 프로젝트가 어떻게 돌아가는지 다 이해하신 겁니다.

강조 여기서 한 박자 쉬어가도 좋습니다. "이 흐름만 기억하시면 됩니다" 부분에서 청중과 눈 맞춤.

▶ 검은 트리와 오른쪽 설명을 매칭시켜 가며

프로젝트 폴더 구조는 굉장히 단순합니다. 파일이 총 6개인데, 그중 **핵심은 단 세 개**입니다.

첫 번째 **app.py**는 사용자와 AI를 연결하는 메인 파일이고, 두 번째 **prompts.py**는 AI에게 답변 규칙을 알려주는 파일, 세 번째 **tools.py**는 AI 답변을 더 똑똑하게 도와주는 보조 파일입니다.

나머지 **style.css**, **requirements.txt**, **README.md**는 디자인이나 설치 정보를 담고 있는 보조 파일이라고 보시면 됩니다. 다음 슬라이드부터 이 세 파일을 하나씩 자세히 설명드리겠습니다.

코드 설명 ① · app.py

▶ 오른쪽 INPUT → 처리 → AI 호출 → OUTPUT 박스를 차례로

첫 번째 파일은 **app.py**입니다. 이 파일은 **사용자와 AI를 잇는 메인 파일**이라고 보시면 됩니다.

역할은 크게 네 가지입니다. 사용자가 입력한 값을 **받고**, 필요한 정보를 **모으고**, AI에게 요청을 **보내고**, 받은 답변을 화면에 다시 **보여주는** 일까지.

오른쪽 박스를 보시면 입력, 처리, AI 호출, 출력 순서대로 진행되는데, 이 네 단계가 전부 app.py 안에서 일어난다고 기억해주시면 됩니다.

코드 설명 ② · prompts.py

▶ 오른쪽 8 LAYERS 카드를 가리키며

두 번째 파일은 **prompts.py**입니다. 이름이 어려워 보이지만, 쉽게 말하면 **AI에게 어떻게 답변할지 알려주는 규칙 모음**입니다.

AI가 **어떤 말투로** 답할지, **어떤 형식으로** 추천할지, 그리고 **절대 하면 안 되는 답변**이 무엇인지를 미리 정의해 둔 파일입니다.

오른쪽을 보시면 총 **8개의 레이어** (= 단계별 규칙)로 나뉘져 있는데요, AI의 역할 정의부터 최종 답변 형식까지 단계별로 지시하도록 설계했습니다. 덕분에 매번 비슷한 톤으로, 같은 형식의 답변이 나옵니다.

코드 설명 ③ · tools.py

▶ 오른쪽 6 FUNCTIONS 카드 그리드를 가리키며

세 번째는 **tools.py**입니다. 이 파일은 **AI 답변을 더 똑똑하게 만드는 보조 함수 모음**입니다.

AI에게 입력을 넘기기 전에, 낱씨 조건을 정리하고, 어울리는 스타일을 판단하고, 검색 결과를 짧게 요약해주는 역할을 합니다.

오른쪽 6가지 함수가 보이실 텐데, 핵심 아이디어는 하나입니다. **"정제된 입력 → 더 정확한 답변"**. **깔끔하게 정리된 입력이 AI에게 전달돼야 답변 품질이 올라간다는** 것입니다.

▶ 가운데 화살표(HTTP POST → / JSON RESPONSE ←)를 왼쪽에서 오른쪽 순으로 짚으며

지금까지 세 개의 파일 역할을 봤다면, 이번 슬라이드는 그 파일들이 AI와 어떻게 대화하는지를 보여드립니다.

왼쪽이 `app.py`, 오른쪽이 **Ollama 서버**인데요, 둘은 같은 컴퓨터 안에 있지만 **HTTP 통신** (= 웹 페이지를 주고받을 때 쓰는 약속된 통신 방식)으로 데이터를 주고받습니다. `requests` (= 파이썬의 HTTP 통신 도구)라는 라이브러리로 `localhost:11434` 주소에 요청을 보냅니다.

보낼 때는 JSON 형식으로 `model · prompt · stream · options` 네 가지를 같이 보내고요. 받을 때는 AI가 만든 답변이 `response` 필드에 담겨서 돌아옵니다. 결국 모든 통신이 인터넷이 아니라 내 PC 안에서 끝난다는 게 핵심입니다.

TIP "localhost"가 어렵게 느껴지면 "내 컴퓨터 안의 주소"라고 한 번 풀어 말해주세요.

▶ 터미널 화면을 보여주며, 8단계는 빠르게 훑어가며

실행 방법은 총 8단계로 정리했습니다. 너무 자세히 들어가지는 않고, 흐름만 빠르게 보여드리면요.

깃에서 코드를 받고, **가상환경** (= 프로젝트 전용 작업 공간)을 만들고, 라이브러리를 설치합니다. 그 다음 Ollama를 설치하고 모델을 받습니다.

마지막으로 터미널 두 개를 띄워서 하나에서는 **Ollama 모델을 실행**하고, 다른 하나에서 **Streamlit을 실행**합니다. 그러면 브라우저에서 `localhost:8501` 주소로 접속해 바로 사용할 수 있습니다.

TIP 코드 한 줄 한 줄 읽지 마세요. "이런 순서로 명령어 몇 줄만 입력하면 끝납니다" 정도로.

장점 & 향후 개선 — 마무리

▶ STRENGTHS와 NEXT를 양손으로 비교 하듯

마지막으로 장점과 앞으로의 계획을 정리하겠습니다.

가장 큰 장점은 **로컬 LLM 기반**이라 인터넷 없이도 내 PC에서 바로 돌아간다는 점, 그리고 **코드 구조를 직접 이해하며 만들 수 있는 학습용 프로젝트**라는 점입니다.

앞으로는 실제 날씨 API를 붙이고, 지역 자동 감지, 옷장 저장 기능, 모바일 대응까지 확장할 계획입니다.

WEATHER FIT TALK는 단순한 챗봇이 아니라, 날씨와 상황을 이해해 사용자에게 실용적인 스타일링을 제안하는 로컬 AI 서비스입니다.

마 무 리 마지막 문장은 또박또박 한 박자 천천히. 그대로 다음 슬라이드(감사합니다)로 자연스럽게 넘어갑니다.

감사합니다 — Q & A

▶ "감사합니다" 슬라이드에서 — 천천히, 환한 표정으로

여기까지가 제가 준비한 발표 내용입니다. **들어주셔서 정말 감사드립니다.**

프로젝트에 대해 궁금하신 점이나 코드 관련 질문 있으시면 편하게 물어봐주세요. 발표 끝나고도 따로 여쭙보셔도 좋습니다.

다시 한 번 감사드립니다.

마 무 리 "감사합니다" 후 1-2초 천천히 멈춤 → 가볍게 목례 → 박수 받기. 너무 급하게 자리로 돌아가지 마세요.